Warehouse Order Fulfillment Simulation

Simulation-Based Decision Making with Neural Networks

EGR 608 Team Project

---

1. Project Overview

This project models an **e-commerce warehouse order fulfillment system** using Discrete-Event Simulation (DES) built in Simio. The core research question is:

> Can a neural network make better picking-path decisions than a hand-crafted rule-based policy?

We generate data from the simulation, train a feedforward neural network offline, deploy it back into the simulation, and compare performance against a rule-based baseline.

---

2. System Design

2.1 System Flow

The system follows a linear flow with a branching decision stage in the middle:

```
Order Intake → Order Sorting → [Decision Point] → Picking Path → Packing → Quality Check → Outbound
```

2.2 Q-Network Structure

Zone	Nodes	Notes
Common Upstream	Order Intake, Order Sorting	All orders pass through here
Decision Point	Picking Mode Selection	Core branching logic
Path A	Picking → Verification → Transfer	Single-item picking
Path B	Batch Picking → Sorting → Verification → Transfer	Multi-order batch picking
Path C	Zone Picking → Consolidation	Zone-relay picking
Common Downstream	Packing → Quality Check → Outbound	All paths merge here

2.3 Shared Resources

Three shared resources create contention across the system:

Resource	Quantity	Scope
Forklift	3 units	Shared by all three picking paths (upstream bottleneck)
Packing Stations	4 stations	Shared downstream after path merge
QC Inspectors	2 inspectors	Shared downstream after packing

The forklift is the key upstream shared resource — even if a path has a short queue, limited forklift availability can still cause delays. This non-linear interaction is what the neural network is expected to learn.

---

3. Process Parameters

3.1 Common Upstream

Node	Processing Time	Capacity
Order Intake	Exp(μ = 3 min)	—
Order Sorting	Tri(1, 2, 4 min)	—
Forklift (shared)	Exp(μ = 5 min)	3 units

3.2 Path A — Single-Item Picking (3 steps)

Node	Processing Time	Resource
Picking	Tri(3, 5, 9 min)	1 worker
Verification	Exp(μ = 2 min)	—
Transfer	Conveyor belt	—

Expected path time ≈ 14 min

3.3 Path B — Batch Picking (4 steps)

Node	Processing Time	Resource
Batch Picking	Tri(4, 7, 12 min)	2 workers
Sorting	Uni(3, 6 min)	1 sorting station
Verification	Exp(μ = 2 min)	—
Transfer	Conveyor belt	—

Expected path time ≈ 18 min

3.4 Path C — Zone Picking (2 steps)

Node	Processing Time	Resource
Zone Picking	Tri(2, 4, 7 min)	3 zone workers
Consolidation	Conveyor belt	—

Expected path time ≈ 8 min

> **Design intent:** Path C is fastest on average but has limited capacity. Path B is slowest but handles multiple orders simultaneously. The optimal choice shifts depending on system load — this makes the decision non-trivial and gives the NN room to outperform rule-based logic.

3.5 Common Downstream

Node	Processing Time	Resource
Packing	Tri(4, 6, 10 min)	4 stations (shared)
Quality Check	Exp(μ = 3 min)	2 inspectors (shared)

---

4. Rework Logic

Quality check failures (15% of orders) trigger a rework loop:

```
Quality Check → FAIL (15%)
    └── rework_count + 1
         ├── rework_count ≤ 2  →  Return to Decision Point (re-pick via A/B/C)
         └── rework_count > 2  →  Force Path A directly (bypass decision point)
```

- The `rework_count` attribute is tracked per entity throughout the simulation.
- Forcing Path A after two failed re-picks models a real-world manual intervention — a dedicated worker handles the problem order without competing through the normal decision process.
- This design means Path A queue can grow longer under high rework load, which the NN must learn to account for.

---

5. Arrival & Simulation Settings

Parameter	Value
Order arrival rate	Poisson, $\lambda = 10$ orders/hr
Data generation run length	200 hr (first 10 hr warm-up discarded)
Deployment evaluation run length	50 hr (separate model)

---

6. Decision Policies

6.1 Rule-Based Policy (Baseline)

Two rule-based policies are implemented for comparison:

Rule 1 — Shortest Queue (lower bound baseline) Select the path with the fewest entities currently waiting:

```
Select argmin(Q_A, Q_B, Q_C)
```

Rule 3 — Forklift-Adjusted Wait Time (main baseline) Estimate wait time per path accounting for forklift contention:

```
forklift_wait = max(0, (Q_A + Q_B + Q_C - F) / 3) × (1/12) hr
W_A = forklift_wait + Q_A × 5 min
W_B = forklift_wait + Q_B × 7 min
W_C = forklift_wait + Q_C × 4 min
Select argmin(W_A, W_B, W_C)
```

where F = number of available forklifts.

6.2 Neural Network Policy

- **Input features (4 dimensions):** Q_A, Q_B, Q_C (queue lengths at each path), F (available forklifts)
- **Output label:** Total time in system (minutes) for each candidate path
- **Decision rule:** Select the path with the smallest predicted total time
- **Training data:** Generated from the data generation simulation model
- **Training:** Performed offline in Python (separate from simulation)
- **Deployment:** Trained model deployed into a separate evaluation simulation model

---

7. Simulation–Learning–Deployment Pipeline

```
[Data Generation Model]          [Offline Training]        [Deployment Model]
  Run simulation (200 hr)   →   Export dataset        →   Load trained NN
  Log (queue lengths,            Train feedforward NN       NN replaces rule-based
  total time in system)          Compare architectures      decision at picking
stage
  Vary system states             Select best model          Evaluate performance
```

---

8. Evaluation Metrics

Metric	Role
Time in system	Primary comparison metric
Throughput	Secondary discussion metric
Average waiting time	Secondary discussion metric
Resource utilization	Secondary discussion metric

Performance is compared across three policies: **Rule 1 (Shortest Queue)**, **Rule 3 (Forklift-Adjusted)**, and **NN-based policy**.

---

9. Key Modeling Assumptions

- All inter-arrival times follow a Poisson process (exponential inter-arrival gaps)
- Service times are stationary (no shift effects or worker fatigue modeled)
- Forklift travel time is absorbed into the Exp(5 min) service time
- Rework count resets to 0 after a successful quality check pass
- The conveyor belt transfer time is assumed negligible (deterministic, near-zero)
- No order cancellations or priority preemption

Built with Simio · Neural network trained in Python · EGR 608 Team Project